

עבודה על פרוייקט מסכם

מודול פייתון

TomerBu

תוכנית להיום:

יצירת משתמשים - users

Permissions

Django JWT Authentication

נקודות קצה להרשמה והתחברות - Auth

הגדרות CORS

יצירת משתמשים - users

<http://127.0.0.1:8000/api/users/>

נרצה לקבוע אילו שדות יוצגו

נרצה לקבוע אילו שדות
יקבעו על ידי המערכת

לדוגמא תאריך רישום - אוטו'
לדוגמא id - אוטו'

(ולכן לא נבקש אותם בעת הרשמה)

```
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

[
  {
    "id": 1,
    "password": "pbkdf2_sha256$1000000$iF25GfxqPCLDhl04A",
    "last_login": "2025-06-24T11:03:24.827400Z",
    "is_superuser": true,
    "username": "itomers",
    "first_name": "",
    "last_name": "",
    "email": "tomербу@gmail.com",
    "is_staff": true,
    "is_active": true,
    "date_joined": "2025-06-24T11:02:41.989756Z",
    "groups": [],
    "user_permissions": []
  },
  ,
]
```

יצירת משתמשים - users

```
class UserSerializer(ModelSerializer):  
    class Meta:  
        model = User  
        fields = ['id', 'email', 'username', 'password']  
        extra_kwargs = {'password': {'write_only': True}}  
        # fields = '__all__'
```

בחרנו שדות (אימייל, סיסמא וכו')

write_only

הסיסמא תתקבל רק בבקשות כתיבה כגון POST/PUT
ולא תוחזר בבקשות קריאה כגון GET

יצירת משתמשים - users

בעיה:

אפשר למלא סיסמא 123456

הפתרון - ביטוי רגולרי:

api/serializers.py

```
from rest_framework import serializers
from django.core.validators import RegexValidator
```

```
class UserSerializer(ModelSerializer):
    password = serializers.CharField(
        validators=[
            RegexValidator(regex=r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[A-Za-z\d]{8,}$',
                           message='Password must contain...')
        ]
    )
```

תוצאה - אי אפשר להזין 123 כסיסמא

יצירת משתמשים - users

בעיה:

הסיסמא נשמרת בדטה-בייס כטקסט (אין הצפנה)

```
{  
    "id": 3,  
    "email": "Joe@gmail.com",  
    "username": "Joe",  
    "password": "123456"  
}
```

פרצת אבטחה

```
def create(self, validated_data):  
    #user = User(**validated_data)  
    user = User.objects.create_user(**validated_data)  
    return user
```

יצירת משתמשים - users

http://127.0.0.1:8000/api/users/3/

בעיה:

טיפלנו בcreate

אבל לא טיפלנו בupdate

בעריכה - הסיסמא לא מוצפנת

User Instance

PUT /api/users/3/

HTTP 200 OK

Allow: GET, PUT, PATCH, DELETE, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "id": 3,
  "email": "Joe@gmail.com",
  "username": "Joe",
  "password": "aaaAAA111"
}
```


ההבדל בין update לבין create

ב-CREATE אנחנו מקבלים את כל התכונות של המשתמש:
email, username, password

update אנחנו יכולים לקבל רשימה חלקית של התכונות ולא את כולן.

אין מתודה מובנית במחלקה user לעדכון משתמש:

יש רק פעולה אחת - save

כדי לעדכן:

נעתיק את כל התכונות לאובייקט מהטיפוס User

נעדכן את הסיסמא - מוצפנת

ואז נקרא לsave

יצירת משתמשים - users

http://127.0.0.1:8000/api/users/3/

```
def update(self, instance:User, validated_data):  
    # create_user לא קיימת מתודה מוכנה כמו  
    # ולכן נצטרך לדאוג להצפנה של הסיסמא - ידנית  
  
    # נחלץ את הסיסמא מהמילון:  
    password = validated_data.pop('password', None)  
  
    # instance הוא אובייקט של User  
    # התכונות מהמילון לאובייקט של המשתמש - בלולאה  
    for key, value in validated_data.items():  
        setattr(instance, key, value)  
  
    # מתודה מובנית במחלקה User  
    # שמצפינה סיסמאות  
    instance.set_password(password)  
  
    instance.save()  
  
    return instance
```

(1) מוציאים את הסיסמא מהמילון ושומרים בצד

בלולאה - רצים על המילון
ומעדכנים את התכונות באובייקט של הUser

מתודה במחלקה user שיודעת להצפין סיסמא ולשמור אותה

נקרא לsave

הסתרה של הסיסמא בGET

```
class UserSerializer(ModelSerializer):
    password = serializers.CharField(
        write_only=True,
        validators=[
            RegexValidator(regex=r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[A-Za-z\d]{8,}$',
                           message="Password must contain at least 8 characters...")
        ]
    )
class Meta:
    model = User
    fields = ['id', 'email', 'username', 'password']
```

מכיוון שדרסנו את הpassword
נצטרך להגדיר write_only מחדש בדריסה

Permissions:

למי מותר לבצע פעולה

Authorization

Authentication:

זיהוי של המשתמש

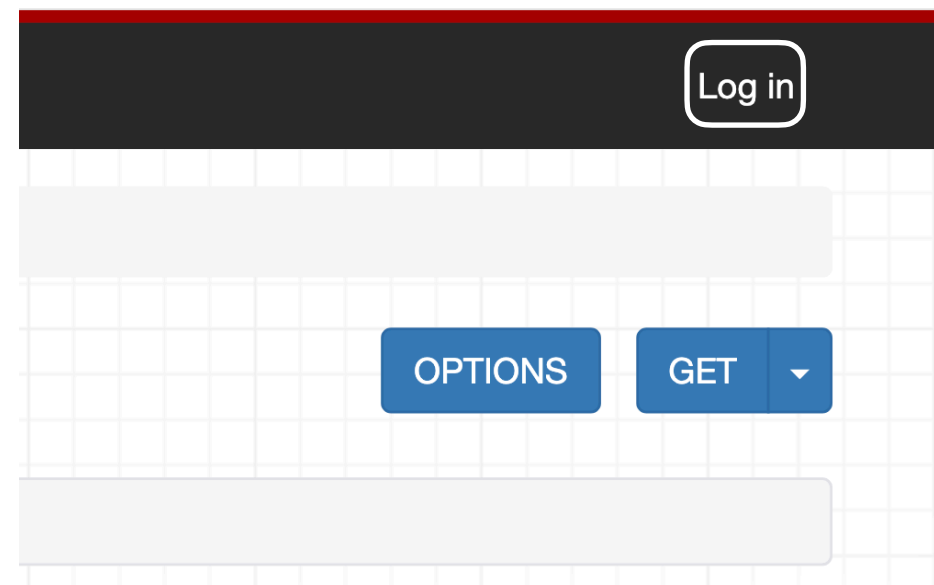
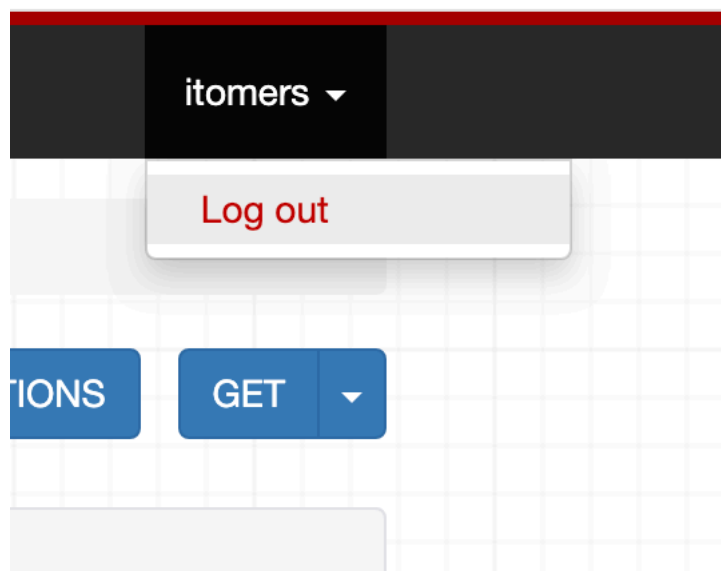
הוספת כפתור התחברות/התנתקות

final/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
    path('api-auth/', include('rest_framework.urls'))
]

# api/ we include all the api module endpoints
```



Permissions:

הגדרת ברירת מחדל - הגדרה גלובלית לפרוייקט

final/settings.py

```
REST_FRAMEWORK = {  
    # permissions:  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticated',  
        # 'rest_framework.permissions.AllowAny',  
    ]  
}
```

נאפשר רק למשתמשים מחוברים לקבל תגובה מהAPI:



גישה לכל סוגי המשתמשים

`rest_framework.permissions.AllowAny`

Permissions:

הגדרת **Per View** - הגדרה פרטנית ל**View**

```
from rest_framework.permissions import AllowAny
```

```
class TagViewSet(ModelViewSet):  
    queryset = Tag.objects.all()  
    serializer_class = TagSerializer  
    permission_classes = [AllowAny]
```

Permissions:

הגדרת **Per View** - הגדרה פרטנית ל**View**

```
from rest_framework.permissions import IsAuthenticatedOrReadOnly
```

```
class TagViewSet(ModelViewSet):  
    queryset = Tag.objects.all()  
    serializer_class = TagSerializer  
    permission_classes = [IsAuthenticatedOrReadOnly]
```

הרשאות קריאה לכולם (כולל משתמש אנונימי)
הרשאות כתיבה למשתמש מחובר בלבד

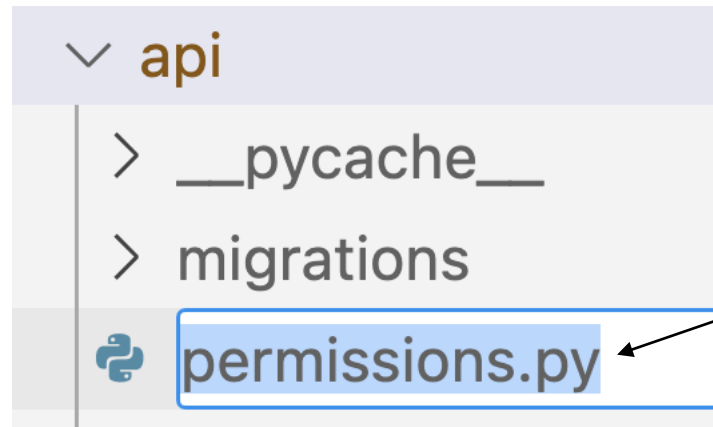
קריאה בלבד:

Tag List

GET /api/tags/

```
HTTP 200 OK  
Allow: GET, POST, HEAD, OPTIONS  
Content-Type: application/json  
Vary: Accept  
  
[  
  {  
    "id": 1,  
    "name": "ASP .Net"  
  },  
  {  
    "id": 2,  
    "name": "Typescript"  
  },  
  {  
    "id": 3,  
    "name": "Django"  
  },  
  {  
    "id": 4,  
    "name": "Django Rest Framework"  
  }  
]
```


ניצור קובץ משלנו להגדרת Permissions



בתוך תיקיית API ניצור קובץ חדש:

```
from rest_framework import permissions

class IsAdmin(permissions.BasePermission):
    """
    Allow only admin users to access the view.
    """
    def has_permission(self, request, view):
        return request.user and request.user.is_staff and request.user.is_superuser

class IsStaff(permissions.BasePermission):
    """
    Allow only admin users to access the view.
    """
    def has_permission(self, request, view):
        #return request.user.userprofile.bio = 'tomer'
        return request.user and request.user.is_staff
```


Django Permissions

מיועד לפעולות קריאה

```
def has_permission(self, request, view):
```

בתוך request יש את user

לפי user אפשר לקבוע - האם הוא מנהל לדוגמא

מיועד לפעולות עריכה ומחיקה

```
def has_object_permission(self, request, view, obj):
```

אנחנו מקבלים את האובייקט עצמו!

לדוגמא אובייקט של Comment

בView

אפשר לבדוק האם המשתמש (מתוך request)

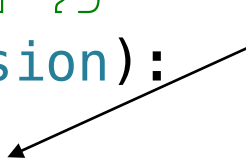
כתב את התגובה - לפי האובייקט של התגובה...

אז ניתן לו רשות

```
# רק למשתמש שכתב את התגובה - מותר לערוך אותה
# כל משתמש יכול לקרוא את כל התגובות - אבל לא לערוך אותן
class CommentOwnerOrReadOnly(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):
        # if the request method is read-only - allow access
        if request.method in permissions.SAFE_METHODS:
            return True

        # the object
        if hasattr(obj, 'author') and hasattr(obj.author, 'user'):
            return obj.author.user == request.user

        return False
```



```
class Comment(models.Model):
    # one to many
    author = models.ForeignKey(
        UserProfile, on_delete=models.CASCADE
    )
```

```
class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, unique=True)
```

```
class PostsPermission(permissions.BasePermission):
    # regular user can only view
    # admin can also edit and delete

    def has_permission(self, request, view):

        # read – allow any:
        if request.method in permissions.SAFE_METHODS:
            return True

        #write – allow admin
        return request.user.is_superuser or request.user.is_staff
```

אותו דבר יהיה לתגיות

הרשאות פרטניות לכל View בפרויקט:

```
from rest_framework import permissions

class IsAdmin(permissions.BasePermission):
    """
    Allow only admin users to access the view.
    """
    def has_permission(self, request, view):
        return request.user and request.user.is_staff and request.user.is_superuser

class CommentOwnerOrReadOnly(permissions.BasePermission):

    def has_object_permission(self, request, view, obj):

        if request.method in permissions.SAFE_METHODS:
            return True

        if hasattr(obj, 'author') and hasattr(obj.author, 'user'):
            return obj.author.user == request.user

        return False

class PostsPermission(permissions.BasePermission):

    def has_permission(self, request, view):

        if request.method in permissions.SAFE_METHODS:
            return True

        return request.user.is_superuser or request.user.is_staff

class TagsPermission(PostsPermission):
    """
    Allow admins to write Tags, all users can read
    """

class UserProfilePermission(permissions.BasePermission):
    """
    allow the user and admin to edit, rest can view
    """
    def has_object_permission(self, request, view, obj):
        return obj.user == request.user or request.user.is_superuser

class UserLikesPermission(permissions.BasePermission):
    """
    allow the user and admin to edit, rest can view
    """
    def has_object_permission(self, request, view, obj):
        return obj.user == request.user or request.user.is_superuser
```

הרשאות פרטניות לכל View בפרויקט:

```
from .permissions import (CommentOwnerOrReadOnly, PostsPermission, IsAdmin,
                          TagsPermission, UserLikesPermission, UserProfilePermission)

class UserViewSet(ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [IsAdmin]

class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all()
    serializer_class = TagSerializer
    permission_classes = [TagsPermission]

class PostUserLikesViewSet(ModelViewSet):
    queryset = PostUserLikes.objects.all()
    serializer_class = PostUserLikesSerializer
    permission_classes = [UserLikesPermission]

class PostViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [PostsPermission]

class UserProfileViewSet(ModelViewSet):
    queryset = UserProfile.objects.all()
    serializer_class = UserProfileSerializer
    permission_classes = [UserProfilePermission]

class CommentViewSet(ModelViewSet):
    queryset = Comment.objects.all()
    serializer_class = CommentSerializer
    permission_classes = [CommentOwnerOrReadOnly]
```

Authentication:

זיהוי של המשתמש

JWT

<https://django-rest-framework-simplejwt.readthedocs.io/en/latest/>

התקנה:

```
pip install djangorestframework-simplejwt
```

final/settings.py

```
REST_FRAMEWORK = {  
    # permissions:  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',  
        # 'rest_framework.permissions.AllowAny',  
    ],  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        # default browsable api authentication  
        'rest_framework.authentication.SessionAuthentication',  
        # JWT authentication  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ],  
}
```

JWT Custom Claims

הוספת מידע אישי לToken

https://django-rest-framework-simplejwt.readthedocs.io/en/latest/customizing_token_claims.html

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView

class BlogTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        # Add custom claims
        token['isadmin'] = user.is_superuser
        # ...

        return token
```

BlogTokenObtainPairSerializer.get_token(user)

JWT Custom Claims

הוספת מידע אישי לToken

core/auth.py

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView

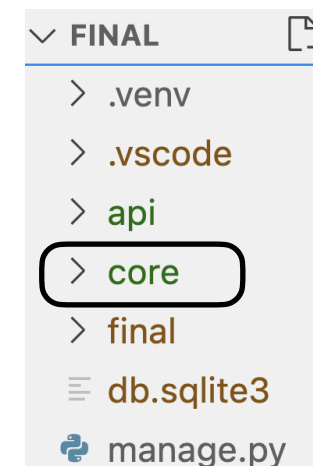
class BlogTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        # Add custom claims
        token['isadmin'] = user.is_superuser
        # ...

        return token

# a function that takes a user object and returns dict/json with jwt
def get_token_for_user(user):
    token = BlogTokenObtainPairSerializer.get_token(user)

    return {
        'jwt': str(token.access_token),
    }
```



נקודות קצה - להרשמה והתחברות:

הירושה מ-ViewSet היא מהסיבה שאין מחלקה מוכנה להרשמה והתחברות:
נצטרך לכתוב 2 מתודות בעצמנו...

ViewSet מאפשר שימוש בראוטר זה נוח

```
from rest_framework.viewsets import ViewSet
from rest_framework.permissions import AllowAny

class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]
```

<https://www.django-rest-framework.org/api-guide/viewsets>

ViewSet להרשמה והתחברות:

נקודות קצה - להרשמה והתחברות:

אפשר לכתוב פונקציות עם שם שאנחנו בחרנו כגון login

```
from rest_framework.viewsets import ViewSet
from rest_framework.permissions import AllowAny
```

```
from rest_framework.decorators import action
```

```
class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]
```

```
@action(methods=['post', 'get'], detail=False)
def login(self, request):
    pass
```

מימוש הרשמה:

קיימת מחלקה מובנית שבדקת שם משתמש וסיסמא ומשווה מול הדטה-בייס:

```
class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]

    @action(methods=['post', 'get'], detail=False)
    def register(self, request):
        serializer = UserSerializer(data=request.data)
        # validation by our rules in UserSerializer and in User:
        # example check that password has at least 8 characters
        serializer.is_valid(raise_exception=True)

        user = serializer.save() # calls the create method
        jwt = get_token_for_user(user)

        # יוצר פרופיל למשתמש אוטומטית בעת ההרשמה
        UserProfile.objects.get_or_create(user = user)
        return Response({'message': 'Registered successfully', 'user':serializer.data, **jwt})
```

מימוש התחברות:

קיימת מחלקה מובנית שבדקת שם משתמש וסיסמא ומשווה מול הדטה-בייס:

```
from rest_framework.auth_token.serializers import AuthTokenSerializer
```

```
# create the serializer object
serializer = AuthTokenSerializer(
    data=request.data, context={'request': request}
)

serializer.is_valid(raise_exception=True) # 401

# get the user from the serializer:
user = serializer.validated_data['user']
```

מימוש התחברות:

קיימת מחלקה מובנית שבודקת שם משתמש וסיסמא ומשווה מול הדטה-בייס:

```
from rest_framework.viewsets import ViewSet
from rest_framework.decorators import action
from core.auth import get_token_for_user
from rest_framework.response import Response
from rest_framework.auth_token.serializers import AuthTokenSerializer

class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]

    @action(methods=['post', 'get'], detail=False)
    def login(self, request):

        # create the serializer object
        serializer = AuthTokenSerializer(
            data=request.data, context={'request': request}
        )

        # if password!=password -> throw
        serializer.is_valid(raise_exception=True) # 401

        # get the user from the serializer:
        user = serializer.validated_data['user']

        jwt = get_token_for_user(user)

        return Response(jwt)
```

מימוש התחברות:

מתודה שמציגה את כל הנתיבים בBrowsable api

כשעובדים עם ModelViewSet
הוא מממש את המתודה הזאת אוטומטית

```
class AuthViewSet(ViewSet):  
    queryset = User.objects.all()  
    serializer_class = UserSerializer  
    permission_classes = [AllowAny]  
  
    def list(self, request):  
        return Response({  
            'login': 'http://127.0.0.1:8000/api/auth/login',  
            'register': 'http://127.0.0.1:8000/api/auth/register',  
        })
```

כשעובדים עם ViewSet
נממש את המתודה list כדי שהפעולות actions
יוצגו יפה בBrowsable api

מימוש התחברות:

urls להרשמה והתחברות:

```
from rest_framework.routers import DefaultRouter

from .views import (CommentViewSet, PostViewSet, PostUserLikesViewSet,
                    UserProfileViewSet, UserViewSet, TagViewSet, AuthViewSet)

router = DefaultRouter()

router.register('tags', TagViewSet, basename='tags')
router.register('auth', AuthViewSet, basename='auth')

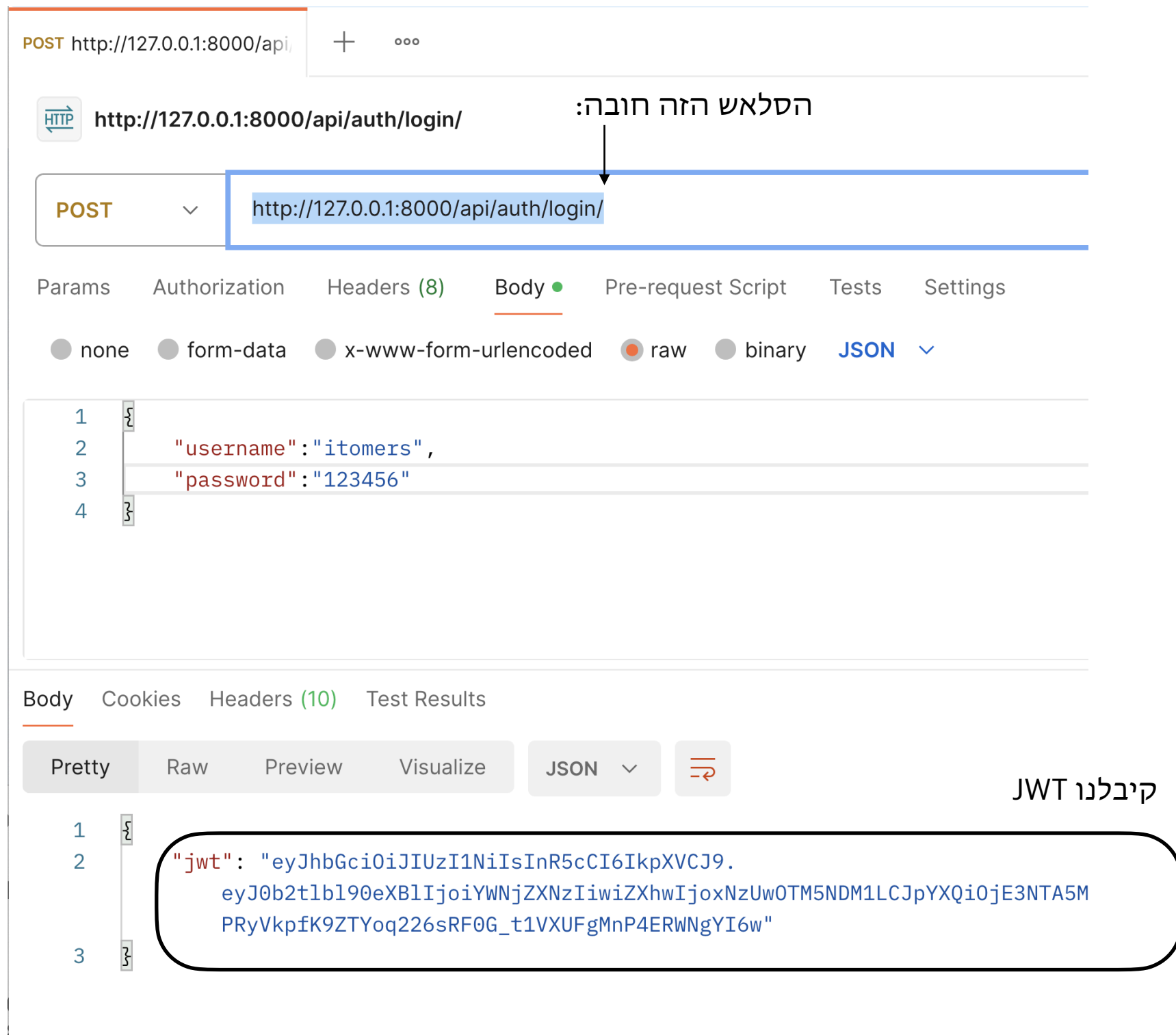
router.register('comments', CommentViewSet, basename='comments')
router.register('posts', PostViewSet, basename='posts')
router.register('users', UserViewSet, basename='users')
router.register('userprofiles', UserProfileViewSet, basename='userprofiles')
router.register('likes', PostUserLikesViewSet, basename='likes')

urlpatterns = router.urls

#optional: we can add more patterns here:
urlpatterns += [

]
```


בקשות ותגובות עם Postman: לוגין



בקשות ותגובות עם Postman

POST http://127.0.0.1:8000/api/ GET http://127.0.0.1:8000/api/ + ...

 http://127.0.0.1:8000/api/users **דוגמא לבקשה שנשלחה בלי header ונחסמת עם סטטוס 403**

GET http://127.0.0.1:8000/api/users

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Headers  6 hidden

	Key
☐	Authorization
	Key

Body Cookies Headers (10) Test Results

Pretty Raw Preview Visualize JSON 

```
1 {
2   "detail": "Authentication credentials were not provided."
3 }
```

בקשות ותגובות עם Postman

The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8000/api/users`. The 'Headers' tab is active, showing a table with one header: 'Authorization' with the value 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp...'.

Key	Value
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp...

Annotations in the image point to the 'Authorization' key and the Bearer token value.

Below the headers, the 'Body' tab is active, showing a JSON response in 'Pretty' format:

```
1 {
2   "id": 1,
3   "email": "tomerbu@gmail.com",
4   "username": "itomers"
5 }
6 {
7   "id": 2,
```

כשיש JWT מקבלים תגובה חיובית: 200 סטטוס:

שם הheader
Authorization

JWT רווח Bearer

התוכן של הJWT שקיבלנו מלוגין

הגדרה - לכמה זמן תקף JWT:

final/settings.py

```
from datetime import timedelta

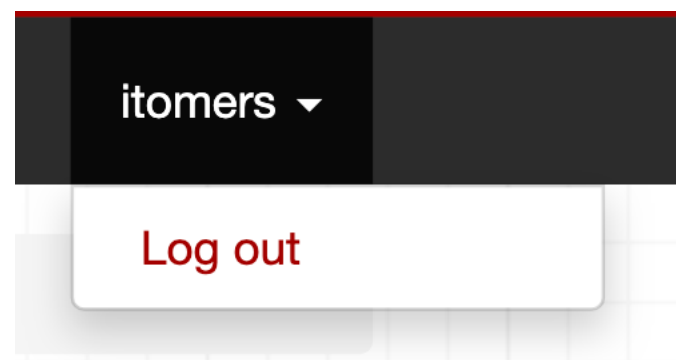
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(days=365)
}
```

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(days=365)
}
```

```
class timedelta(
    days: float = ...,
    seconds: float = ...,
    microseconds: float = ...,
    milliseconds: float = ...,
    minutes: float = ...,
    hours: float = ...,
    weeks: float = ...
```

המידע של המשתמש צריך להגיע מהJWT ולא נתון לבחירה:

<http://127.0.0.1:8000/api/comments>



Raw data

HTML form

Text

Nice Read

Author

moe Profile

itomers

Post

Typescript by itomers

Reply to

POST

המידע של המשתמש צריך להגיע מהJWT ולא נתון לבחירה:

<https://www.django-rest-framework.org/api-guide/fields/#default>

שדות עם ערך ברירת מחדל מיוחד: כמו user

core/auth.py

```
class CurrentUserDefault:
    """
    May be applied as a `default=...` value on a serializer field.
    Returns the current user.
    """
    requires_context = True

    def __call__(self, serializer_field):
        return serializer_field.context['request'].user

class CurrentProfileDefault:
    """
    May be applied as a `default=...` value on a serializer field.
    Returns the current user.
    """
    requires_context = True

    def __call__(self, serializer_field):
        return serializer_field.context['request'].user.userprofile
```

המידע של המשתמש צריך להגיע מהJWT ולא נתון לבחירה:

api/serializers.py

```
from rest_framework.fields import HiddenField, SerializerMethodField
from core.auth import CurrentProfileDefault, CurrentUserDefault

class CommentSerializer(ModelSerializer):
    # take the author from the jwt:
    author = HiddenField(default = CurrentProfileDefault())

    # add the author_id to the json:
    author_id = SerializerMethodField('get_author_id')
    class Meta:
        model = Comment
        fields = "__all__"

    # a helper method that returns the id of the author:
    def get_author_id(self, obj):
        return obj.author.id
```

המידע של הauthor הוא שדה default:
מתקבל מהJWT

כדי להציג את הid של הauthor שנבחר מהJWT
נוסיף שדה בעצמנו `author_id`

שדה שמתודה מחשבת לנו:

אותו דבר בדיוק Posts

api/serializers.py

```
class PostSerializer(ModelSerializer):
    author = HiddenField(default = CurrentProfileDefault())
    author_id = SerializerMethodField('get_author_id')
    # a helper method that returns the id of the author:
    def get_author_id(self, obj):
        return obj.author.id

    class Meta:
        model = Post
        fields = "__all__"
```


אותו דבר כמעט בPostUserLikes

איפה שכתוב author נשנה לuser (כי ככה קוראים לזה במודל)

api/serializers.py

```
class PostUserLikesSerializer(ModelSerializer):
    user = HiddenField(default=CurrentProfileDefault())
    user_id = SerializerMethodField('get_user_id')
    # a helper method that returns the id of the user:

    def get_user_id(self, obj):
        return obj.user.id

    class Meta:
        model = PostUserLikes
        fields = "__all__"
```


כמעט אותו דבר ב UserProfileSerializer

api/serializers.py

```
class UserProfileSerializer(ModelSerializer):
    user = HiddenField(default=CurrentUserDefault())
    user_id = SerializerMethodField('get_user_id')
    # a helper method that returns the id of the user:

    def get_user_id(self, obj):
        return obj.user.id

    class Meta:
        model = UserProfile
        fields = "__all__"
```

כאן יש לנו user ב request
ולא user profile

כי זה ה user profile

:CORS

מה הכתובת של צד לקוח
למי מותר להשתמש בשרת

<https://pypi.org/project/django-cors-headers/>

```
pip install django-cors-headers
```

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'api',  
    'rest_framework',  
    'corsheaders',  
]
```

:CORS

מה הכתובת של צד לקוח
למי מותר להשתמש בשרת

settings.py

```
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'corsheaders.middleware.CorsMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.auth.middleware.AuthenticationMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  
]
```

יש חשיבות לסדר - זה חייב להיות לפני הCommonMiddleware

:CORS

מה הכתובת של צד לקוח
למי מותר להשתמש בשרת

settings.py

```
CORS_ALLOW_ALL_ORIGINS = True

CORS_ALLOWED_ORIGINS = [
    "https://example.com",
    "https://sub.example.com",
    "http://localhost:8080",
    "http://127.0.0.1:9000",
    "http://127.0.0.1:3000",
    "http://127.0.0.1:5173",
]
```

לקוח מכתובת מוכרת: יקבל תגובה

לקוח מכתובת לא מוכרת: לא יקבל תגובה

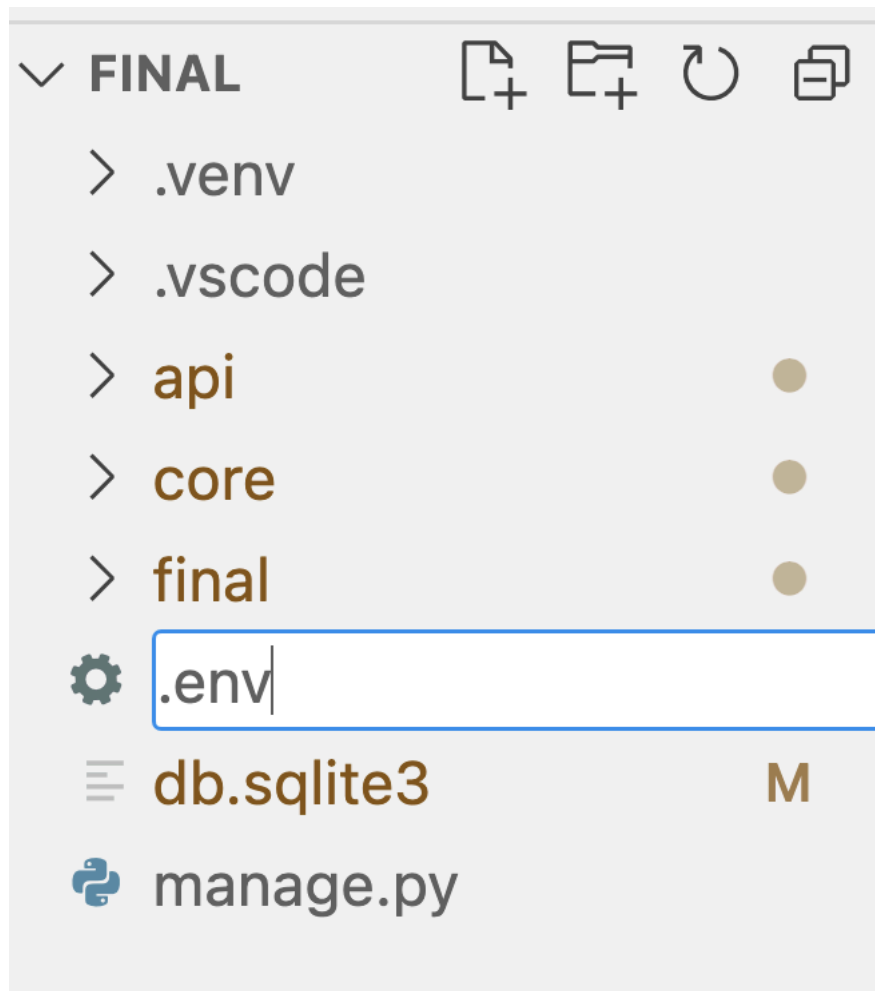
עבודה עם משתני סביבה:

```
pip install python-decouple
```

1

ניצור קובץ .env
בתיקייה הראשית

2



⚙️ .env

×

נמלא את הקובץ

3

⚙️ .env

1

DB_NAME=b log

עבודה עם משתני סביבה:

```
from decouple import config
# process.env.DB_NAME
dbn = config('DB_NAME')

print(dbn)
```